

Apellidos:

Nombre:

Matrícula:

Concurrencia (parte 2)/turno B (Solución)

Curso 2020–2021 - 2º Semestre (Junio 2021) Grado en Ingeniería

Informática / Grado en Matemáticas e Informática /
Doble Grado en Ing. Informática y ADE

UNIVERSIDAD POLITÉCNICA DE MADRID

Respuestas

1	2	3	4	5	6

NORMAS: Este es un cuestionario que consta de 7 preguntas. Todas las preguntas son de respuesta simple excepto la pregunta 7 que es una pregunta de desarrollo. La puntuación total del examen es de 10 puntos. La duración total es de una hora. Rellenad **apellidos, nombre y número de matrícula** en todas las hojas.

El test debe contestarse en la caja de respuestas de la esquina superior derecha. Sólo hay una respuesta válida a cada pregunta de respuesta simple. Toda pregunta en que se marque más de una respuesta se considerará incorrectamente contestada. Toda pregunta incorrectamente contestada restará del examen una cantidad de puntos igual a la puntuación de la pregunta dividido por el número de alternativas ofrecidas en la misma.

Cuestionario

(1 punto) 1. Considere el siguiente código:

<pre>class P1 implements CProcess{ public void run(){ ch2.out().write(null); ch1.out().write(null); System.out.print("A"); } }</pre>	<pre>class P2 implements CProcess{ public void run(){ System.out.print("C"); ch2.in().read(); ch3.in().read(); } }</pre>	<pre>class P3 implements CProcess{ public void run(){ ch1.in().read(); ch3.out().write(null); System.out.print("B"); } }</pre>
<pre>Any2OneChannel ch1 = Channel.any2one(); Any2OneChannel ch2 = Channel.any2one(); Any2OneChannel ch3 = Channel.any2one(); public static final void main(final String[] args){ new Parallel (new CProcess[] {new P1(), new P2(), new P3()}).run(); }</pre>		

Se pide señalar la respuesta correcta.

- (a) ☒ La salida del programa sólo podrá ser CAB o CBA.
 (b) ☐ La salida del programa siempre será CAB.
 (c) ☐ El programa se podría quedar bloqueado y no siempre acabaría.

(1 punto) 2. Dado un recurso compartido implementado con JCSP. Se pide señalar la respuesta correcta:

- (a) ☒ Cuando `fairSelect(sCond)` devuelve un valor `sel`, siempre se cumplirá que `sCond[sel] == true`.
 (b) ☐ Cuando `fairSelect(sCond)` devuelve un valor `sel`, no siempre se cumplirá que `sCond[sel] == true`.

(1 punto) 3. Dado el siguiente CTAD:

ACCIÓN op1:
 ACCIÓN op2:
 TIPO: $\text{MiCTAD} = \mathbb{Z}$
 INICIAL: $\text{self} = 0$
 CPRE: $\text{self} \% 2 = 0$
 op1()
 POST: $\text{self} = \text{self}^{\text{pre}} + 1$
 CPRE: $\text{self} \% 2 = 1$
 op2()
 POST: $\text{self} = (\text{self}^{\text{pre}} + 1) \% 2$

NOTA: Como habitualmente, la operación % es el resto de la división entera (módulo).

Se ha decidido implementarlo con monitores mediante el siguiente código:

<pre> public class MiCTAD{ private Monitor mutex = new Monitor(); private Monitor.Cond c = mutex.newCond(); private int self = 0; public void op1(){ mutex.enter(); if (self % 2 != 0) {c.await();} self = self+1; desbloquear(); mutex.leave(); } </pre>	<pre> public void op2(){ mutex.enter(); if (self % 2 == 0) {c.await();} self = (self+1) % 2; desbloquear(); mutex.leave(); } private void desbloquear () { if (self % 2 == 0) { c.signal(); } else if (self % 2 == 1) { c.signal(); } } </pre>
--	---

Se pide señalar la respuesta correcta.

- (a) ☒ Podrían ejecutarse operaciones cuya CPRE no se cumple.
 (b) ☐ Se trata de una implementación correcta del recurso compartido.

(1 punto) 4. Dada esta otra implementación del CTAD de la pregunta 3.

<pre> public class MiCTAD{ private Monitor mutex = new Monitor(); private Monitor.Cond c1 = mutex.newCond(); private Monitor.Cond c2 = mutex.newCond(); private int self = 0; public void op1(){ mutex.enter(); if (self % 2 != 0) {c1.await();} self = self+1; c2.signal(); mutex.leave(); } </pre>	<pre> public void op2(){ mutex.enter(); if (self % 2 == 0) {c2.await();} self = (self+1) % 2; c1.signal(); mutex.leave(); } </pre>
---	--

Se pide marcar la afirmación correcta:

- (a) ☒ Se trata de una implementación correcta del recurso compartido
 (b) ☐ Podría darse el caso de que hubiese hilos esperando en c1 o en c2 que, pudiendo ejecutarse, no se desbloqueen.

(1 punto) 5. Dada esta implementación mediante paso de mensajes del CTAD de la pregunta 3.

```
class P implements CSProcess {
    Any2OneChannel petOp1 = Channel.any2one();
    Any2OneChannel petOp2 = Channel.any2one();

    public void run() {
        int self = 0;
        final int OP1 = 0;
        final int OP2 = 1;
        final Guard[] entradas =
            {petOp1.in(), petOp2.in()};
        final Alternative servicios =
            new Alternative (entradas);
        final boolean[] sincCond =
            new boolean[2];

        while (true) {
            sincCond[OP1] = self % 2 == 0;
            sincCond[OP2] = self % 2 == 1;
            int sel = servicios.fairSelect(sincCond);
            switch (sel) {
                case OP1:
                    petOp1.in().read();
                    self = self + 1;
                    break;
                case OP2:
                    petOp2.in().read();
                    self = (self + 1) % 2;
                    break;
            }
        }
    }

    public void op1() {
        petOp1.out().write(null);
    }

    public void op2() {
        petOp2.out().write(null);
    }
}
```

Se pide señalar la respuesta correcta.

- (a) ☐ Podrían ejecutarse operaciones cuya CPRE no se cumple.
(b) ☒ Se trata de una implementación correcta del recurso compartido.

- (1½ puntos) 6. Se define una clase *servidor* P y tres clases *cliente* P1, P2 y P3 que se comunican a través de los canales de comunicación petA, petB y petC (se muestran las partes relevantes del código para este problema).

<pre> class P implements CSProcess { Any2OneChannel petA = Channel.any2one(); Any2OneChannel petB = Channel.any2one(); Any2OneChannel petC = Channel.any2one(); public void run() { boolean q = true; boolean r = false; boolean s = true; final int A = 0; final int B = 1; final int C = 2; final Guard[] entradas = {petA.in(), petB.in(), petC.in()}; final Alternative servicios = new Alternative(entradas); final boolean[] sincCond = new boolean[3]; while (true) { sincCond[A] = q && !r; sincCond[B] = r && !s; sincCond[C] = s && !q; int sel = servicios.fairSelect(sincCond); switch (sel) { case A: petA.in().read(); q = !q; r = !r; System.out.print("A"); break; case B: petB.in().read(); r = !r; s = !s; System.out.print("B"); break; case C: petC.in().read(); s = !s; q = !q; System.out.print("C"); break; } } } } </pre>	<pre> class P1 implements CSProcess { public void run() { while (true) { petA.out().write(null); } } } class P2 implements CSProcess { public void run() { while (true) { petB.out().write(null); } } } class P3 implements CSProcess { public void run() { while (true) { petC.out().write(null); } } } </pre>
--	---

Dado un programa concurrente con cuatro procesos p, p1, p2 y p3 de las clases P, P1, P2 y P3.

Se pide marcar cuál de las siguientes afirmaciones es la correcta:

- (a) ☒ El programa ejecutará indefinidamente y la salida por consola será ACBACBACB. . .
- (b) ☐ El programa imprimirá A y los 4 procesos acabarán bloqueados.
- (c) ☐ Es seguro que los cuatro procesos van a ejecutar indefinidamente, pero no se puede saber con qué salida concreta.

(3½ puntos) 7. A continuación mostramos la especificación formal de un recurso gestor *MiCTAD*.

C-TAD *MiCTAD*

OPERACIONES

ACCIÓN uno:

ACCIÓN dos: $Indice[e]$

ACCIÓN tres:

TIPO: $MiCTAD = Indice \rightarrow \mathbb{B}$

TIPO: $Indice = \{0, 1\}$

INICIAL: $\forall i \in Indice \bullet \neg self(i)$

CPRE: $\neg self(0) \vee \neg self(1)$

uno()

POST: $self(0) = Cierito \wedge self(1) = Cierito$

CPRE: $self(v)$

dos(v)

POST: $self = self^{pre} \oplus \{0 \mapsto \neg self^{pre}(0)\}$

CPRE: $\neg self(0) \wedge self(1)$

tres()

POST: $self(0) = \neg self^{pre}(0) \wedge self(1) = \neg self^{pre}(1)$

Se pide: Completar la implementación de este recurso mediante monitores. En cuanto al código de desbloques podéis optar tanto por un método de desbloqueo genérico como por tener código de desbloqueo especializado en los distintos métodos. Si optáis por la segunda posibilidad dejad en blanco el cuerpo del método desbloqueo.

```
class MiCTad {

    private boolean [] self;
    private Monitor mutex;

    private Condition cuno;
    private Condition [] cdos;
    private Condition ctres;

    public MiCTad() {
        indice = new boolean[]{false,false};
        mutex = new Monitor ();
        cuno = mutex.newCond();
        cdos[] = new Condition[] { mutex.newCond(), mutex.newCond() };
        ctres = mutex.newCond();
    }
    public void uno() {
        mutex.enter();
        if (self[0] && self[1]) {
            cuno.await();
        }

        self[0] = true;
        self[1] = true;

        desbloqueo();

        mutex.leave();
    }
    public void dos(int v) {
        mutex.enter();
        if (!self[v]) {
            cdos[v].await();
        }

        self[0] = !self[0];

        desbloqueo();
        mutex.leave();
    }
    public void tres() {
        mutex.enter();
        if (self[0] || !self[1]) {
            ctres.await();
        }

        self[0] = !self[0];
        self[1] = !self[1];

        desbloqueo();
        mutex.leave();
    }
}
```

Apellidos:

Nombre:

Matrícula:

```
private void desbloqueo() {
    boolean signaled = false;
    if ((!self[0] || !self[1]) && cuno.waiting() > 0) {
        cuno.signal();
        signaled = true;
    }
    for (int v = 0; v < cdos.length && !signaled; v++) {
        if (self[v] && cdos[v].waiting() > 0) {
            cdos[v].signal();
            signaled = true;
        }
    }
    if ((!self[0] && self[1]) && ctres.waiting() > 0 && !signaled) {
        ctres.signal();
    }
}
```